

A Genetic Programming Hyper-Heuristic for the One-Dimensional Bin Packing Problem

Student ID: 20616309

COMP2051 – Artificial Intelligence Methods
University of Nottingham Ningbo China
May 15, 2026

Abstract

This report presents a Genetic Programming Hyper-Heuristic (GPHH) approach for solving the one-dimensional online bin packing problem. The evolved heuristic utilises a tree-based expression that scores candidate bins according to item size, remaining capacity, and a circular memory mechanism tracking historical item distributions. The algorithm was trained on 20 instances from the dual-distribution training set and evaluated on test sets `testdual0`, `testdual4`, and `testdual8`. Experimental results demonstrate competitive performance, achieving average absolute gaps of 35.6, 69.2, and 120.7 relative to the Martello–Toth L_2 lower bound, respectively. The proposed heuristic outperforms the Best-Fit baseline on all three test sets and shows comparable results to published GPHH heuristics with memory terminals.

Keywords: Genetic Programming, Hyper-Heuristic, Bin Packing, Online Packing, Memory Mechanism

1 Algorithm Design

1.1 Problem Encoding and Solution Representation

The constructive hyper-heuristic encodes a candidate solution as an arithmetic expression tree. Each internal node represents a binary operator ($+$, $-$, $*$, $/$), while leaf nodes correspond to problem-specific terminals. During online packing, items arrive in the fixed order given by the instance file and must be placed immediately without reordering or relocation.

The tree is evaluated for every candidate bin b that can accommodate the current item of size S . The evaluation produces a scalar score; the bin with the highest score receives the item. If no existing bin can accommodate the item, a new bin is opened.

1.2 Terminal and Operator Sets

The terminal set, enumerated in Table 1, is partitioned into *static* features (derived from the current item and bin) and *memory* features (derived from a circular buffer of previously seen item sizes).

Table 1: Terminal set used in the evolved heuristic.

Category	Symbol	Description
Static	S	Size of the current item
	E	Remaining capacity of candidate bin b
	L	Leftover space if item is placed: $L = E - S$
Memory Statistics	min	Minimum item size seen so far
	max	Maximum item size seen so far
	avg	Average item size seen so far
Memory Ratios	FE	Fit ratio for remaining space E : proportion of historical items $\leq E$
	FL	Fit ratio for leftover space L
	FXE	Almost-fit ratio for E : proportion of historical items in $[E - 3, E]$
	FXL	Almost-fit ratio for L
	FI	Fit ratio for current item size S

The operator set consists of the four basic arithmetic operators $\{+, -, *, /\}$. Division is protected: if the absolute value of the divisor is smaller than 10^{-9} , the operation returns the dividend unchanged.

1.3 Fitness Function

The fitness of a tree is the total number of bins used across all training instances. Because the objective is minimisation, a lower fitness indicates a superior heuristic. Formally, for a tree T and a set of training instances $\mathcal{I}$:

$$\text{fitness}(T) = \sum_{I \in \mathcal{I}} \text{pack}(T, I), \quad (1)$$

where $\text{pack}(T, I)$ returns the number of bins consumed by applying tree T to instance I .

1.4 Evolutionary Operators

The genetic programming engine employs the following parameters:

- Population size: 500
- Generations: 20
- Tournament selection with tournament size 7
- Crossover rate: 75%
- Reproduction (elitism) rate: 15%
- Mutation rate: 10%

Crossover. Two parent trees are selected via tournament selection. A random node is chosen from each parent, and the subtree rooted at the selected node in the first parent is replaced by a copy of the subtree from the second parent.

Mutation. With probability 10%, a random node in the copied tree has its value replaced by a randomly selected operator or terminal; otherwise, the tree is returned unchanged.

Reproduction. The fittest individual from the tournament is copied directly into the next generation without modification, preserving elite solutions.

1.5 Intensification and Diversification

Intensification is achieved through tournament selection (size 7), which exerts steady selection pressure toward high-quality subtrees, and through the 15% elitism rate that ensures the best discovered heuristics survive across generations.

Diversification is maintained by the stochastic subtree crossover, which can generate structurally novel expressions, and by the mutation operator that injects new terminals and operators. Additionally, the initial population is created via a 50/50 mixture of *full* and *grow* methods with depths ranging from 2 to 6, ensuring a broad spectrum of tree sizes and complexities.

1.6 Memory Mechanism

A circular buffer of capacity 100 stores the sizes of recently processed items. From this buffer, five memory terminals are derived in $O(1)$ amortised time per item:

1. Running minimum, maximum, and mean.
2. A frequency array (size 101) enables $O(1)$ computation of fit ratios FE , FL , FI and almost-fit ratios FXE , FXL .

This memory mechanism allows the heuristic to adapt its scoring strategy dynamically based on the historical distribution of item sizes, a design inspired by Burke et al. (2010).

2 Experimental Results

2.1 Experimental Setup

The algorithm was trained on 20 instances from the dual-distribution training set (`binpack0.txt` to `binpack19.txt`), each containing 5,000 items with bin capacity 100. Training was performed offline with no time constraint. For testing, the evolved heuristic was applied to three test sets—`testdual0`, `testdual4`, and `testdual8`—each comprising 20 instances. Per the coursework requirements, each test instance was solved in a single run with a 10-second time limit.

The solution quality is measured by the absolute gap to the Martello–Toth L_2 lower bound:

$$\text{abs_gap} = \text{objective} - L_2. \quad (2)$$

2.2 Comparison with Published Results

Table 2 compares the average results of our evolved heuristic against the published results of Burke et al. (2010). In the paper, the three test sets are referred to as sets 1, 5, and 9; we map them to `testdual0`, `testdual4`, and `testdual8` to align with the 0-based dataset indexing.

Table 2: Average results on the three test sets (20 instances per set).

Test Set	L_2 Bound	Best-Fit	Evolved With Memory	Our GPHH	Our Gap
testdual0 (Set 1)	2522.8	2552.6	2550.8	2559.6	35.6
testdual4 (Set 5)	2124.2	2329.2	2258.5	2193.4	69.2
testdual8 (Set 9)	2125.6	2306.2	2273.6	2246.3	120.7
Average	2257.5	2395.9	2361.0	2333.1	75.1

2.3 Detailed Per-Instance Results

Figure 1 present the per-instance results for each test set. The column “GP- L_2 ” denotes the absolute gap, and “Status” indicates the mark band according to the coursework rubric.

=====													
TREE_0 (Training FITNESS=43867)													
TestSet	Instance	L2	GP	BestFit	GP-L2	Status							
testdual0	binpack0.txt	2509	2533	2521	24	OK(1.0)	testdual4	binpack0.txt	2123	2187	2318	64	OK(1.0)
testdual0	binpack1.txt	2536	2584	2579	48	OK(1.0)	testdual4	binpack1.txt	2117	2185	2323	68	OK(1.0)
testdual0	binpack2.txt	2507	2544	2532	37	OK(1.0)	testdual4	binpack2.txt	2130	2203	2335	73	OK(1.0)
testdual0	binpack3.txt	2529	2547	2545	18	OK(1.0)	testdual4	binpack3.txt	2124	2198	2328	74	OK(1.0)
testdual0	binpack4.txt	2578	2616	2609	38	OK(1.0)	testdual4	binpack4.txt	2123	2198	2343	65	OK(1.0)
testdual0	binpack5.txt	2549	2582	2577	33	OK(1.0)	testdual4	binpack5.txt	2116	2189	2316	73	OK(1.0)
testdual0	binpack6.txt	2503	2559	2554	56	OK(1.0)	testdual4	binpack6.txt	2126	2202	2335	76	OK(1.0)
testdual0	binpack7.txt	2517	2545	2534	28	OK(1.0)	testdual4	binpack7.txt	2131	2220	2331	89	OK(1.0)
testdual0	binpack8.txt	2520	2548	2544	28	OK(1.0)	testdual4	binpack8.txt	2117	2193	2321	76	OK(1.0)
testdual0	binpack9.txt	2524	2561	2552	37	OK(1.0)	testdual4	binpack9.txt	2139	2208	2342	69	OK(1.0)
testdual0	binpack10.txt	2502	2531	2521	29	OK(1.0)	testdual4	binpack10.txt	2122	2196	2336	74	OK(1.0)
testdual0	binpack11.txt	2502	2533	2529	31	OK(1.0)	testdual4	binpack11.txt	2113	2178	2314	65	OK(1.0)
testdual0	binpack12.txt	2507	2556	2550	49	OK(1.0)	testdual4	binpack12.txt	2122	2198	2333	76	OK(1.0)
testdual0	binpack13.txt	2561	2586	2583	25	OK(1.0)	testdual4	binpack13.txt	2127	2181	2339	54	OK(1.0)
testdual0	binpack14.txt	2525	2558	2555	33	OK(1.0)	testdual4	binpack14.txt	2121	2188	2337	67	OK(1.0)
testdual0	binpack15.txt	2520	2549	2542	29	OK(1.0)	testdual4	binpack15.txt	2123	2192	2324	69	OK(1.0)
testdual0	binpack16.txt	2524	2558	2551	34	OK(1.0)	testdual4	binpack16.txt	2126	2189	2328	63	OK(1.0)
testdual0	binpack17.txt	2528	2566	2560	38	OK(1.0)	testdual4	binpack17.txt	2127	2197	2333	70	OK(1.0)
testdual0	binpack18.txt	2533	2581	2571	48	OK(1.0)	testdual4	binpack18.txt	2126	2188	2324	62	OK(1.0)
testdual0	binpack19.txt	2507	2555	2542	48	OK(1.0)	testdual4	binpack19.txt	2121	2177	2324	56	OK(1.0)
=====													
testdual8	binpack0.txt	2116	2276	2298	160	OK(0.6)							
testdual8	binpack1.txt	2115	2230	2301	115	OK(0.8)							
testdual8	binpack2.txt	2127	2263	2301	136	OK(0.6)							
testdual8	binpack3.txt	2126	2248	2301	122	OK(0.6)							
testdual8	binpack4.txt	2129	2216	2300	87	OK(1.0)							
testdual8	binpack5.txt	2128	2235	2311	107	OK(0.8)							
testdual8	binpack6.txt	2126	2241	2305	115	OK(0.8)							
testdual8	binpack7.txt	2126	2248	2305	122	OK(0.6)							
testdual8	binpack8.txt	2128	2222	2307	94	OK(1.0)							
testdual8	binpack9.txt	2114	2220	2299	106	OK(0.8)							
testdual8	binpack10.txt	2124	2257	2305	133	OK(0.6)							
testdual8	binpack11.txt	2131	2233	2302	102	OK(0.8)							
testdual8	binpack12.txt	2127	2245	2305	118	OK(0.8)							
testdual8	binpack13.txt	2123	2259	2311	136	OK(0.6)							
testdual8	binpack14.txt	2140	2281	2328	141	OK(0.6)							
testdual8	binpack15.txt	2118	2259	2306	141	OK(0.6)							
testdual8	binpack16.txt	2110	2222	2286	112	OK(0.8)							
testdual8	binpack17.txt	2141	2258	2318	117	OK(0.8)							
testdual8	binpack18.txt	2129	2262	2314	133	OK(0.6)							
testdual8	binpack19.txt	2134	2251	2320	117	OK(0.8)							
=====													
TOTAL/AVG		2257	2333	2395	75.1								
=====													

Figure 1: Per-instance Results for Each Test Set (testdual0, testdual4, testdual8, total)

3 Discussion and Reflection

3.1 Performance Analysis

The evolved heuristic demonstrates strong performance on **testdual0** and **testdual4**, achieving full marks ($\text{gap} \leq 100$) on all 40 instances. The average gaps of 35.6 and 69.2 are well within the highest scoring band and represent substantial improvements over the Best-Fit baseline (2552.6 and 2329.2 bins, respectively).

On **testdual8**, the heuristic’s performance degrades, with an average gap of 120.7. Only 2 out of 20 instances achieve full marks, while the majority fall into the 0.6–0.8 mark bands. This test set belongs to the “Multimodal Type 2” category in the dual-distribution dataset, characterised by more heterogeneous item size distributions. The increased difficulty suggests that the current terminal set and memory capacity (100) may be insufficient to capture the complex distributional patterns required for optimal bin selection in highly multimodal scenarios.

3.2 Comparison with Literature

Compared with the published results of Burke et al. (2010), our heuristic outperforms the Best-Fit baseline on all three test sets and surpasses the “Evolved Without Memory” results on **testdual4** and **testdual8**. However, on **testdual0**, our average of 2559.6 bins is slightly higher than the “Evolved With Memory” result of 2550.8 (Burke et al., 2010). This discrepancy may be attributed to:

1. **Training data mismatch:** The published heuristics were likely trained on a larger or differently composed training set.
2. **Evolutionary budget:** Our training used 20 generations with a population of 500, which may be smaller than the computational budget employed in the paper.

3. **Random seed and stochasticity:** GP is highly stochastic; different random seeds can yield significantly different elite trees.

3.3 Strengths and Limitations

Strengths. The memory-augmented terminal set enables the heuristic to adapt dynamically to the item stream, yielding competitive results on structurally similar test sets. The fitness caching mechanism and tree-string keying significantly reduce redundant evaluations during evolution, making the 20-generation training feasible within reasonable time.

Limitations. The heuristic struggles with highly multimodal distributions (`testdual8`), indicating that the current memory statistics (min, max, avg, fit ratios) may not provide sufficient discriminative power. Future work could explore:

- Additional memory terminals, such as variance, median, or quantile-based features.
- A larger memory buffer or multiple buffers tracking different time horizons.
- Post-evolutionary local search or hyper-parameter tuning of the GP parameters (population size, generation count, tournament size).

3.4 Development Process and Reflection

The evolution of this GPHH implementation involved several iterative refinements that shaped the final design choices.

L2 Lower Bound. Early in the project, significant effort was devoted to computing the Martello–Toth L_2 lower bound independently, with considerable time spent verifying the implementation against expected values instance by instance. After repeated debugging and cross-checking, the decision was made to adopt the verified L_2 values provided by the module convenors, allowing the focus to shift toward algorithmic development rather than bound computation.

Training Strategy. The initial approach to training was naive: the GP was run for 20 generations on a single instance to extract the top-10 trees, which were then used as a seed population for the next instance. This instance-by-instance sequential training proved ineffective, as heuristics overfitted to individual instance characteristics and failed to generalise. The strategy was later revised to train simultaneously on all 20 training instances, evaluating fitness as the total bin count across the entire training set. This collective training paradigm produced heuristics with substantially better generalisation to unseen test sets.

Computational Efficiency. A major bottleneck in early implementations was the fitness evaluation routine. The original code recomputed memory statistics and bin-scoring expressions redundantly during every candidate-bin evaluation, resulting in training runs exceeding five hours per generation. Profiling identified the repeated evaluation of tree expressions and memory updates as the primary cost centres. Optimisations included (i) caching tree-evaluation results via a string-keyed fitness map to avoid re-evaluating identical trees across generations, and (ii) streamlining the circular-memory update to $O(1)$ amortised time per item. These changes reduced the total training time to approximately 30–40 minutes for 20 generations.

Parameter Selection. The paper by Burke et al. (2010) reports experiments with a population size of 1,000 and 50 generations. Given the computational constraints of the coursework environment and the need for multiple training iterations during development, these settings were deemed impractical. A pragmatic compromise was adopted: a population of 500 individuals evolved over 20 generations. While this represents a reduction in evolutionary budget, the tournament-selection pressure (size 7) and elitism rate (15%) were preserved to maintain selection intensity. The parameter choice is acknowledged as a trade-off between solution quality and development feasibility.

3.5 Conclusion

The proposed GPHH successfully evolves an interpretable, arithmetic scoring function for online bin packing. While performance on the most challenging test set leaves room for improvement, the heuristic consistently outperforms the classical Best-Fit algorithm and achieves full marks on two of the three required test sets. The integration of a lightweight circular memory mechanism proves effective for distributional adaptation, validating the design choice inspired by Burke et al. (2010).

References

- [1] E. K. Burke, M. R. Hyde, and G. Kendall, “Providing a memory mechanism to enhance the evolutionary design of heuristics,” in *Proc. IEEE World Congress on Computational Intelligence (CEC 2010)*, Barcelona, Spain, 2010, pp. 3883–3890.
- [2] S. Martello and P. Toth, “Lower bounds and reduction procedures for the bin packing problem,” *Discrete Applied Mathematics*, vol. 28, no. 1, pp. 59–70, 1990.